

CSC495 Project Report: Defect Prediction in Python Repositories

Introduction

Identifying defect-prone modules early in the software development life cycle is critical in reducing maintenance costs and improving software quality. This project aims to develop and evaluate an automated tool that predicts risky Python files within Git repositories. The prediction is driven by quantifiable historical metrics derived from bug-fixing commits, enabling proactive detection of potential defects.

Methodology

The methodology begins with mining the Git history of selected repositories over a defined period as shown in the configuration file. Bug-fixing commits are identified using a lightweight keyword-based classifier that flags commit messages likely related to defect resolution. The tool isolates modified non-test Python files from these commits and calculates six key historical metrics using PyDriller: recency of changes, frequency of modifications, number of lines modified, lines of code, cyclomatic complexity, and number of minor contributors. Each metric serves as a proxy for risk. For instance, high frequency suggests instability, while larger code size and complexity imply greater maintenance difficulty. These metrics are then normalized to a uniform scale between 0 and 1 to compute a final risk score for each file using a weighted linear combination of these normalized metrics.

Weight Optimization & Caching

To determine the optimal contribution of each metric to defect prediction, we implemented two weight optimization strategies. The first was extreme value testing, where each weight set gave one metric a higher weight than the others. The second was absence testing, where each weight set removed one metric entirely. Several predefined weight sets, stored in `weightsets.csv`, were tested across four repositories using configurations from `config1.json` to `config4.json`. Each repository had an associated ground truth of known risky files defined in `optimize_ground_truth.csv`. To make this process computationally efficient, we introduced a caching mechanism. Normalized metric values for each repository were calculated once and stored using the `get_normalized_metrics` function. These cached values were then reused during repeated risk score evaluations with different weight sets via the `calculate_risk_from_cached` function. This significantly reduced redundant computation and cut down PyDriller runs from 28 to just 4.

Results & Insights

Through iterative weight optimization and evaluation using the test harness, recency consistently proved to be the most influential factor in predicting risky files. Configurations heavily weighting recency performed best, and its removal significantly degraded accuracy. Frequency of modification ranked second in importance. While metrics like cyclomatic complexity, LOC, lines modified, and minor contributors provided some signal, their overall impact on accuracy was less significant.

Based on the results, we increased the weight assigned to recency from 0.225 to 0.45 and adjusted others accordingly. This targeted recalibration resulted in a notable improvement in average accuracy across the test repositories, rising from approximately 0.66 to 0.74. This finding suggests that how recently and how often a file changes (historical metrics) can be more predictive of defects than its inherent structural complexity. The efficiency of this optimization process was critically dependent on our caching strategy. Pre-calculating and storing metrics for each repository avoided redundant computations during weight testing, significantly accelerating the evaluation cycles and enabling a more thorough exploration of the weight space.